Méthodologie d'Audit de Sécurité

Guide technique et pratique — en français

Ce document présente une méthodologie complète d'audit de sécurité informatique, expliquée de manière technique mais accessible. Il est destiné aux étudiants, développeurs et professionnels qui souhaitent comprendre comment auditer un système de bout en bout, avec des exemples concrets à chaque étape.

Auteur : OgonasSec — Étudiant en Réseau et Sécurité Informatique (UVCI)

Date : Octobre 2026

Table des matières

1. Introduction — Qu'est-ce qu'un audit de sécurité ?
2. Phase 1 : Reconnaissance (OSINT)
3. Phase 2 : Scanning et énumération
4. Phase 3 : Analyse des vulnérabilités
5. Phase 4 : Exploitation (preuve de concept)
6. Phase 5 : Post-exploitation
7. Phase 6 : Rapport et recommandations
8. Outils essentiels
9. Exemple complet d'audit
10. Conclusion et bonnes pratiques

1. Introduction — Qu'est-ce qu'un audit de sécurité ?

Un audit de sécurité est un processus systématique d'évaluation d'un système informatique pour identifier ses faiblesses (vulnérabilités) avant qu'un attaquant ne les exploite. L'objectif n'est pas de "casser" le système, mais de le renforcer.

Un audit se décompose en 6 phases principales :

- Reconnaissance (OSINT) — collecter des informations publiques
- Scanning et énumération — découvrir les services et ports ouverts
- Analyse des vulnérabilités — identifier les failles connues
- Exploitation — prouver qu'une faille est exploitable (PoC)
- Post-exploitation — évaluer l'impact réel
- Rapport — documenter et recommander des corrections

Note importante : un audit doit toujours être réalisé avec l'autorisation écrite du propriétaire du système. Sans autorisation, c'est du hacking illégal.

2. Phase 1 : Reconnaissance (OSINT)

La reconnaissance consiste à collecter un maximum d'informations sur la cible sans interagir directement avec elle. On utilise des sources ouvertes (Open Source Intelligence).

Techniques courantes :

- Recherche DNS : trouver les sous-domaines (ex. api.exemple.com, admin.exemple.com)
- Recherche WHOIS : obtenir le registrar, les contacts techniques, les serveurs DNS
- Google Dorks : recherches avancées pour trouver des fichiers exposés, des pages d'admin
- Réseaux sociaux : LinkedIn, GitHub pour trouver des employés et des technologies utilisées
- Shodan/Censys : moteurs de recherche pour les appareils connectés à Internet

Exemple concret :

Cible : example.com. On utilise `dig example.com ANY` pour lister les enregistrements DNS. On trouve un sous-domaine `staging.example.com` qui n'est pas protégé. On le note pour la phase de scanning.

Outils : dig, nslookup, whois, theHarvester, Shodan, Google Dorks.

3. Phase 2 : Scanning et énumération

Le scanning consiste à interagir avec la cible pour découvrir les services actifs, les ports ouverts et les versions des logiciels.

Étapes :

- Scan de ports : identifier quels ports sont ouverts (TCP/UDP)
- Détection de services : déterminer quel logiciel tourne sur chaque port
- Détection de versions : obtenir la version exacte du service

- Énumération : lister les utilisateurs, partages, répertoires, etc.

Exemple concret :

On lance `nmap -sV -sC 192.168.1.10` sur la cible. Résultat : port 22 (SSH, OpenSSH 8.2), port 80 (HTTP, Apache 2.4.41), port 3306 (MySQL 5.7). On note les versions pour chercher des vulnérabilités connues.

Outils : nmap, masscan, nikto, dirb, gobuster, enum4linux.

4. Phase 3 : Analyse des vulnérabilités

L'analyse consiste à croiser les informations collectées avec des bases de vulnérabilités connues (CVE, CWE) pour identifier les failles potentielles.

Sources de vulnérabilités :

- CVE (Common Vulnerabilities and Exposures) : base de données mondiale des failles
- CWE (Common Weakness Enumeration) : classification des types de faiblesses
- Exploit-DB : exploits publics disponibles
- OWASP Top 10 : les 10 failles web les plus critiques
- NVD (National Vulnerability Database) : scores CVSS pour évaluer la gravité

Exemple concret :

On trouve Apache 2.4.41. On cherche dans la NVD et on découvre CVE-2021-41773 (path traversal, score CVSS 7.5). Le service est vulnérable. On note la faille pour la phase d'exploitation.

Outils : searchsploit, nuclei, Nessus, OpenVAS, OWASP ZAP.

5. Phase 4 : Exploitation (preuve de concept)

L'exploitation consiste à prouver qu'une vulnérabilité est réellement exploitable. On ne casse pas le système — on démontre la faille de manière contrôlée.

Règles d'or :

- Toujours avoir une autorisation écrite avant d'exploiter
- Ne jamais détruire de données ou perturber le service
- Documenter chaque étape de l'exploitation
- S'arrêter dès que la preuve est obtenue (pas de surexploitation)

Exemple concret :

Pour CVE-2021-41773 (path traversal sur Apache), on envoie une requête HTTP : `GET /cgi-bin/.%2e/%2e%2e/%2e%2e/etc/passwd HTTP/1.1`. Si le serveur renvoie le contenu de /etc/passwd, la faille est confirmée. On capture la réponse comme preuve.

Outils : Metasploit, Burp Suite, SQLmap, Hydra, scripts personnalisés.

6. Phase 5 : Post-exploitation

La post-exploitation évalue l'impact réel de la faille : que peut faire un attaquant une fois à l'intérieur ?

Questions à se poser :

- Peut-on escalader les privilèges (user → root/admin) ?
- Peut-on accéder à d'autres machines du réseau ?
- Peut-on exfiltrer des données sensibles ?
- Peut-on persister (installer un backdoor) ?

Exemple concret :
On a obtenu un shell sur le serveur web. On vérifie les privilèges avec `whoami`. On trouve un fichier de configuration contenant des identifiants de base de données. On peut accéder à la base de données et lire les données clients. Impact : critique.

7. Phase 6 : Rapport et recommandations

Le rapport est la livraison finale de l'audit. Il doit être clair, structuré et actionnable pour l'équipe technique et la direction.

Structure d'un bon rapport :

- Résumé exécutif (pour la direction, en langage simple)
- Périmètre et méthodologie
- Vulnérabilités trouvées (classées par gravité : critique, haute, moyenne, basse)
- Pour chaque faille : description, preuve (screenshot/log), impact, recommandation
- Conclusion et plan de remédiation

Exemple de classification :
CVE-2021-41773 (path traversal) → Gravité : Critique (CVSS 7.5). Impact : lecture de fichiers sensibles. Recommandation : mettre à jour Apache vers la version 2.4.51 ou supérieure.

8. Outils essentiels

Phase	Outils	Usage
Reconnaissance	dig, whois, theHarvester, Shodan	Collecte d'informations
Scanning	nmap, masscan, nikto, gobuster	Découverte de services
Vulnérabilités	Nessus, OpenVAS, nuclei, searchsploit	Détection de failles
Exploitation	Metasploit, Burp Suite, SQLmap, Hydra	Preuve de concept
Post-exploitation	LinPEAS, WinPEAS, BloodHound	Escalade de privilèges
Rapport	Markdown, PDF, Jupyter Notebook	Documentation

9. Exemple complet d'audit

Voici un exemple simplifié d'audit sur une application web fictive (volontairement vulnérable) pour illustrer la méthodologie.

Cible : application web de gestion de comptes (http://cible.local)

Reconnaissance : On découvre que le site utilise WordPress 5.8 et un plugin de formulaire.

Scanning : nmap révèle les ports 80 (HTTP) et 3306 (MySQL). dirb trouve /admin et /backup.

Vulnérabilités : CVE-2021-29447 (XXE dans WordPress media library, CVSS 5.5). SQL injection dans le formulaire de login.

Exploitation : On injecte `' OR 1=1 --` dans le champ login → accès au compte admin. On uploade un shell via le plugin.

Post-exploitation : On accède à la base de données → 500 comptes clients avec emails et mots de passe en clair.

Rapport : 4 vulnérabilités : 1 critique (SQLi), 1 haute (XXE), 1 moyenne (backup exposé), 1 basse (version WordPress obsolète).

10. Conclusion et bonnes pratiques

- Toujours obtenir une autorisation écrite avant tout audit
- Documenter chaque étape pour que le rapport soit reproductible
- Ne jamais exploiter au-delà de la preuve de concept
- Classer les failles par gravité (CVSS) pour prioriser les corrections
- Proposer des recommandations concrètes et réalisables
- Faire un suivi (re-test) après correction des failles
- Respecter la confidentialité des données trouvées

> Un bon audit ne se mesure pas au nombre de failles trouvées, mais à la qualité du rapport et à la capacité de l'équipe à corriger les problèmes identifiés.